Iterable في Java

ملخص مختصر: شو هو، كيف بيشتغل، وكيف تحذف منه بأمان.

الفكرة

`Iterable` واجهة (interface) بتخلي الكلاس تبعك قابل للمرور عليه بحلقة `for-each`. فيها ميثود وحدة بس:

```
public class Library implements Iterable<Book> {
    private List<Book> books = new ArrayList<>();

    @Override
    public Iterator<Book> iterator() {
        return Collections.unmodifiableList(books).iterator();
    }
}

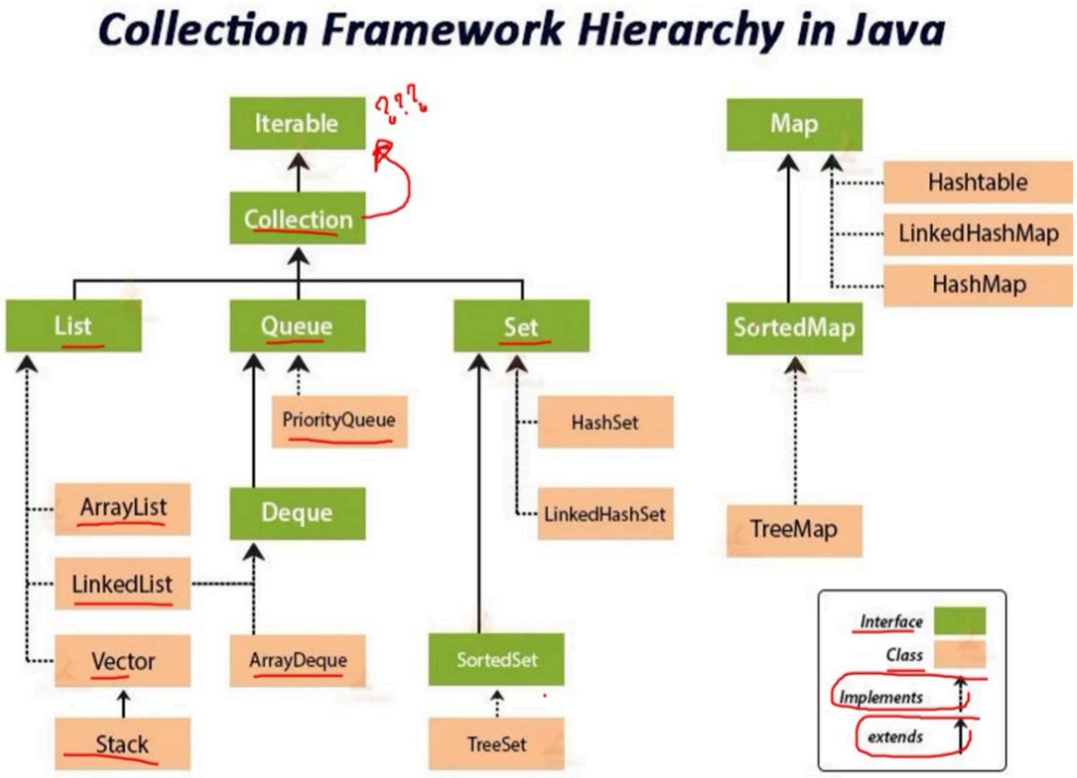
for (Book b : library) System.out.println(b);
```

Iterable مقابل Iterator

	Iterable	Iterator
المعنى	الكائن قابل للمرور عليه	المؤشر اللي بيمر
الميثودات	`iterator()`	`hasNext()` · `next()` · `remove()`
الاستخدام	`for-each`	حلقة `while` يدوية

حلقة `for-each` هي `Iterator` من ورا الستارة: Java بتنادي `iterator()` ثم `hasNext()` و `next()` بدالك.

مكان Iterable في هرم Collections



هرم Collection Framework في Java

أخضر: Interface برتقالي: Class خط متصل: `extends` خط منقط: `implements`

جواب علامات الاستفهام: `Iterable` في أعلى الهرم، و `Collection` بترث منها (`extends`). يعني كل شي تحت `Collection` هو `Iterable` تلقائياً: `List` و `Queue` و `Set` وكل كلاساتهم. هذا سبب إن `ArrayList` و `HashSet` بيشتغلوا مع `for-each` بدون ما تكتب شي.

الواجهة	مميزاتها	أمثلة
`List`	مرتبة وتقبل التكرار	ArrayList · LinkedList · Vector · Stack
`Queue`	طابور (دخول وخروج)	PriorityQueue · ArrayDeque
`Set`	بدون تكرار	HashSet · LinkedHashSet · TreeSet

`Map`	مفتاح وقيمة، مو Iterable	HashMap · LinkedHashMap · TreeMap · Hashtable

- `Map` منفصلة عن الهرم، فما بتقدر تكتب `for (x : map)`. بتمر على `map.keySet()` أو `map.values()` أو `map.entrySet()`.
- `Stack` بترث من `Vector` (خط متصل)، و`LinkedList` بتنفذ `List` و`Deque` معاً.
- في مشروعك، `Library` بتنفذ `Iterable` مباشرة لأنك بس بدك المرور، وما بتحتاج كل ميثودات `Collection` مثل `add` و`remove` المكشوفة.

الحذف الآمن أثناء المرور

```
Iterator<Book> it = books.iterator();
while (it.hasNext()) {
    Book b = it.next();              // مرة وحدة فقط
    if (b.getTitle().toLowerCase().contains(title.toLowerCase())
            && b.getCach() == cach) {
        it.remove();                 // مو books.remove(b)
    }
}
```

ليش تستخدمه

- **تغليف:** ما بتكشف القائمة الداخلية بـ `getBooks()`.
- **مرونة:** بتغير `List` إلى `Set` وكود `Main` ما بيتأثر.
- **تحكم:** الكلاس هو المسؤول عن قواعد الحذف والإضافة.
- **توافق:** بيشتغل مع `for-each` ومع الـ Streams.

الأخطاء الشائعة

الخطأ	السبب	الحل

`UnsupportedOperationException`	قائمة من `Arrays.asList`	`new ArrayList<>(...)`
`ConcurrentModificationException`	`list.remove()` داخل `for-each`	`it.remove()`
`IllegalStateException`	`remove()` قبل `next()`	استدعِ `next()` أولاً
`NoSuchElementException`	`next()` مرتين بدورة وحدة	`Book b = it.next()`
طباعة `Book@eed1f14`	ما في `toString()`	أضف `toString()`

قواعد للتذكر

1. `next()` مرة وحدة بكل دورة، وخزّن النتيجة بمتغير.
2. الحذف أثناء المرور = `it.remove()` فقط.
3. القائمة `private` وبدون getter.
4. استخدم `unmodifiableList` لمنع الحذف من برّا.
5. أي كلاس بدك تطبعه لازم فيه `toString()`.

الخطوة الجاية: اكتب `Iterator` خاص فيك من الصفر.